

P0009R9

mdspan: A Non-Owning Multidimensional Array Reference

Draft Proposal, 2019-01-20

This version:

wg21.link/P0009r9

Issue Tracking:

[GitHub](https://github.com)

Authors:

[H. Carter Edwards](#)

[Bryce Adelstein Lelbach](#)

[Daniel Sunderland](#)

[David Hollman](#)

[Christian Trott](#)

[Mauro Bianco](#)

[Ben Sander](#)

[Athanasios Iliopoulos](#)

[John Michopoulos](#)

[Mark Hoemmen](#)

Audience:

LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/ORNL/cpp-proposals-pub/blob/master/P0009/P0009.bs

Table of Contents

1	Revision History
1.1	P0009r9: Pre 2019-02-Kona Mailing
1.2	P0009r8: Pre 2018-11-SanDiego Mailing
1.3	P0009r7: Post 2018-06-Rapperswil Mailing
1.4	P0009r6 : Pre 2018-06-Rapperswil Mailing
1.5	P0009r5 : Pre 2018-03-Jacksonville Mailing
1.6	P0009r4 : Pre 2017-11-Albuquerque Mailing
1.7	P0009r3 : Post 2016-06-Oulu Mailing
1.8	P0009r2 : Pre 2016-06-Oulu Mailing
1.9	P0009r1 : Pre 2016-02-Jacksonville Mailing
1.10	P0009r0 : Pre 2015-10-Kona Mailing
1.11	Related Activity
2	Description
3	Editing Notes
4	Wording
5	Next Steps
6	Related Work

References

Informative References

§ 1. Revision History

§ 1.1. P0009r9: Pre 2019-02-Kona Mailing

- Wording fixes based on guidance: [LWG small group at 2018-11-SanDiego](#)

§ 1.2. P0009r8: Pre 2018-11-SanDiego Mailing

- Refinement based upon updated [prototype](#) / reference implementation

§ 1.3. P0009r7: Post 2018-06-Rapperswil Mailing

- wording reworked based on guidance: [LWG review at 2018-06-Rapperswil](#)
- usage of `span` requires reference to C++20 working draft
- namespace for library TS `std::experimental::fundamentals_v3`

§ 1.4. P0009r6 : Pre 2018-06-Rapperswil Mailing

P0009r5 was not taken up at 2018-03-Jacksonville meeting. Related [LEWG review of P0900 at 2018-03-Jacksonville meeting](#)

LEWG Poll: We want the ability to customize the access to elements of `span` (ability to restrict, etc):

```
span<T, N, Accessor=...>
```

SF	F	N	A	SA
1	1	1	2	8

LEWG Poll: We want the customization of `basic_mdspan` to be two concepts `Mapper` and `Accessor` (akin to `Allocator` design).

```
basic_mdspan<T, Extents, Mapper, Accessor>
mdspan<T, N...>
```

SF	F	N	A	SA
3	4	5	1	0

LEWG Poll: We want the customization of `basic_mdspan` to be an arbitrary (and potentially user-extensible) list of properties.

```
basic_mdspan<T, Extents, Properties...>
```

SF	F	N	A	SA
1	2	2	6	2

Changes from P0009r5 due to related LEWG reviews:

- Replaced variadic property list with *extents*, *layout mapping*, and *accessor* properties.
- Incorporated [P0454r1](#).
 - Added accessor policy concept.
 - Renamed `mdspan` to `basic_mdspan`.
 - Added a `mdspan` alias to `basic_mdspan`.

§ 1.5. P0009r5 : Pre 2018-03-Jacksonville Mailing

[LEWG review of P0009r4 at 2017-11-Albuquerque meeting](#)

LEWG Poll: We should be able to index with `span<int type[N]>` (in addition to array).

SF	F	N	A	SA
2	11	1	1	0

Against comment - there is not a proven needs for this feature.

LEWG Poll: We should be able to index with 1d `mdspan`.

SF	F	N	A	SA
0	8	7	0	0

LEWG Poll: We should put the requirement on "rank() <= N" back to "rank()==N".

Unanimous consent

LEWG Poll: With the editorial changes from small group, plus the above polls, forward this to LWG for Fundamentals v3.

Unanimous consent

Changes from P0009r4:

- Removed nullptr constructor.
- Added constexpr to indexing operator.
- Indexing operator requires that `rank()==sizeof...(indices)`.
- Fixed typos in examples and moved them to appendix.
- Converted note on how extensions to access properties may cause reference to be a proxy type to an "see below" to make it normative.

§ 1.6. P0009r4 : Pre 2017-11-Albuquerque Mailing

[LEWG review at 2017-03-Kona meeting](#)

[LEWG review of P0546r1 at 2017-03-Kona meeting](#)

LEWG Poll: Should we have a single template that covers both single and multi-dimensional spans?

SF	F	N	A	SA
1	6	2	6	3

Changes from P0009r3:

- Align with P0122r5 span [proposal](#).
- Rename to `mdspan`, multidimensional span, to align with `span`.
- Move preferred array extents mechanism to appendix.
- Expose codomain as a `span`.
- Add layout mapping concept.

§ 1.7. P0009r3 : Post 2016-06-Oulu Mailing

[LEWG review at 2016-06-Oulu](#)

LEWG did not like the name `array_ref`, and suggested the following alternatives: - `sci_span` - `numeric_span` - `multidimensional_span` - `multidim_span` - `mdspan` - `md_span` - `vla_span` - `multispan` - `multi_span`

LEWG Poll: Are member `begin()`/`end()` still good?

SF	F	N	A	SA
0	2	4	3	1

LEWG Poll: Want this proposal to provide range-producing functions outside `array_ref`?

SF	F	N	A	SA
0	1	3	2	3

LEWG Poll: Want a separate proposal to explore iteration design space?

SF	F	N	A	SA
9	1	0	0	0

Changes from P0009r2:

- Removed iterator support; a future paper will be written on the subject.
- Noted difference between multidimensional array versus language's array-of-array-of-array...
- Clearly describe requirements for the embedded type aliases (`element_type`, `reference`, etc).
- Expanded description of how the variadic properties list would work.
- Stopped allowing `array_ref<T[N]>` in addition to `array_ref<extents<N>>`.
- Clarified domain, codomain, and domain -> codomain mapping specifications.
- Consistently use *extent* and *extents* for the multidimensional index space.

§ 1.8. P0009r2 : Pre 2016-06-Oulu Mailing

LEWG review at 2016-02-Jacksonville.

Changes from P0009r1:

- Adding details for extensibility of layout mapping.
- Move motivation, examples, and relaxed incomplete array type proposal to separate papers.
 - P0331: Motivation and Examples for Polymorphic Multidimensional Array.
 - P0332: Relaxed Incomplete Multidimensional Array Type Declaration.

§ 1.9. P0009r1 : Pre 2016-02-Jacksonville Mailing

LEWG review at 2015-10-Kona.

LEWG Poll: What should this feature be called?

Name	#
<code>view</code>	5
<code>span</code>	9
<code>array_ref</code>	6
<code>slice</code>	6
<code>array_view</code>	6
<code>ref</code>	0
<code>array_span</code>	7
<code>basic_span</code>	1
<code>object_span</code>	3
<code>field</code>	0

LEWG Poll: Do we want 0-length static extents?

SF	F	N	A	SA

3	4	2	3	0
---	---	---	---	---

LEWG POLL: Do we want the language to support syntaxes like `X[3][][][5]`?

Syntax	#
<code>view<int[3][0][][5], property1></code>	12
<code>view<int, dimension<3, 0, dynamic_extent, 5>, property1></code>	4
<code>view<int[3][0][dynamic_extent][5], property1></code>	5
<code>view<int, 3, 0, dynamic_extent, 5, property1></code>	4
<code>view<int, 3, 0, dynamic_extent, 5, properties<property1>></code>	2
<code>view<arr<int, 3, 0, dynamic_extent, 5>, property1></code>	4
<code>view<int[3][0][][5], properties<property1>></code>	9

LEWG POLL: Do we want the variadic property list in template args (either raw or in `properties<>`)? Note there is no precedence for this in the library.

SF	F	N	A	SA
3	6	3	0	0

LEWG POLL: Do we want the per-view bounds-checking knob?

SF	F	N	A	SA
3	4	1	2	1

Changes from P0009r0:

- Renamed `view` to `array_ref`.
- How are users allowed to add properties? Needs elaboration in paper.
- `view<int[][][]>::layout` should be named.
- Rename `is_regular` (possibly to `is_affine`) to avoid overloading the term with the `Regular` concept.
- Make static `span()`, `operator()`, constructor, etc variadic.
- Demonstrate the need for improper access in the paper.
- In `operator()`, take integral types by value.

§ 1.10. P0009r0 : Pre 2015-10-Kona Mailing

Original non-owning multidimensional array reference (`view`) paper with motivation, specification, and examples.

§ 1.11. Related Activity

Related [LEWG review of P0546r1 at 2017-11-Albuquerque meeting](#)

LEWG Poll: `span` should specify the dynamic extent as the element type of the first template parameter rather than the (current) second template parameter

SF	F	N	A	SA
5	3	2	2	0

LEWG Poll: `span` should support the addition of access properties variadic template parameters

SF	F	N	A	SA
0	10	1	5	0

Authors agreed to bring a separate paper ([\[P0900r0\]](#)) discussing how the variadic properties will work.

§ 2. Description

The proposed polymorphic multidimensional array reference (`mdspan`) defines types and functions for mapping multidimensional indices in its *domain*, a multidimensional index space, to the `mdspan`'s *codomain*, elements of a contiguous span of objects. A *multidimensional index space* of rank `r` is the Cartesian product

$[0, N_0) \times [0, N_1) \times \dots \times [0, N_{R-1})$ of half-open integer intervals. A *multidimensional index* is a element of a multidimensional index space. An `mdspan` has two policies: the *layout mapping* and the *accessor*. The layout mapping specifies the formula, and properties of the formula, for mapping a multidimensional index from the domain to an element in the codomain. The accessor is an extension point that allows modification of how elements are accessed. For example, [P0367](#) proposed a rich set of potential access properties.

A multidimensional array is not an array-of-array-of-array-of...

The multidimensional array abstraction has been fundamental to numerical computations for over five decades. However, the C/C++ language provides only a one-dimensional array abstraction which can be composed into array-of-array-of-array-of... types. While such types have some similarity to multidimensional arrays, they do not provide adequate multidimensional array functionality of this proposal. Two critical functionality differences are (1) multiple dynamic extents and (2) polymorphic mapping of multidimensional indices to element objects.

Optimized Implementation of Layout Mapping

We intend the layout mapping of a multidimensional index to be a constant-time `constexpr` operation that is trivially inlined and optimized when possible. Compiler vendors may apply optimizations such as loop invariant code motion, including partial evaluation of multidimensional index layout mappings when indices are loop invariant.

§ 3. Editing Notes

The proposed changes are relative to the working draft of the standard as of [N4750](#).

The  character is used to denote a placeholder section number, table number, or paragraph number which the editor shall determine.

Add the header `<mdspan>` to the "C++ library headers" table in **[headers]**, in a place that respects the table's current alphabetic order.

Add the header `<mdspan>` to the "Containers library summary" table in **[containers.general]** below the listing for `<span>`.

§ 4. Wording

- Text in blockquotes is not proposed wording.
- The  character is used to denote a placeholder section number which the editor shall determine.
- Copy the entire **[views]** subclause from the current draft, since it is needed for `basic_mdspan`.

✧ ✧ ✧

Add the following paragraphs to **[views.general]**, after the current text, which consists only of the following sentence: "The header `<span>` defines the view span."

◆. The header `<mdspan>` defines the view `basic_mdspan`, the type alias `mdspan`, and other facilities for interacting with these views.

✧ ✧ ✧

*Add the following subclauses to the end of the **[views]** subclause in the current draft:*

26.7.◆ Header `<mdspan>` synopsis `[mdspan.syn]`

```

namespace std {
namespace experimental {
inline namespace fundamentals_v3 {
    // [mdspan.extents], class template extents
    template<ptrdiff_t... StaticExtents>
        class extents;

    // [mdspan.layout], Layout mapping policies
    class layout_left;
    class layout_right;
    class layout_stride;

    // [mdspan.accessor.basic]
    template<class ElementType>
        class accessor_basic;

    // [mdspan.basic], class template mdspan
    template<class ElementType,
            class Extents,
            class LayoutPolicy = layout_right,
            class AccessorPolicy = accessor_basic<ElementType>>
        class basic_mdspan;

    template<class T, ptrdiff_t... Extents>
        using mdspan = basic_mdspan<T, extents<Extents...>>;

    // [mdspan.extents.compare], extents comparison operators
    template<ptrdiff_t... LHS, ptrdiff_t... RHS>
        constexpr bool operator==(const extents<LHS...>& lhs, const extents<RHS...>& rhs) noexcept;

    template<ptrdiff_t... LHS, ptrdiff_t... RHS>
        constexpr bool operator!=(const extents<LHS...>& lhs, const extents<RHS...>& rhs) noexcept;

    // [mdspan.subspan], subspan creation
    template<class ElementType, class Extents, class LayoutPolicy,
            class AccessorPolicy, class... SliceSpecifiers>
    struct mdspan_subspan { // exposition only
        using extents_t = see below;
        using layout_t = see below;
        using type = basic_mdspan<ElementType, extents_t, layout_t,
                                typename AccessorPolicy::offset_policy>;
    };

    template<class ElementType, class Extents, class LayoutPolicy,
            class AccessorPolicy, class... SliceSpecifiers>
    using mdspan_subspan_t = // exposition only
        typename mdspan_subspan<ElementType, Extents, LayoutPolicy,
                                AccessorPolicy, SliceSpecifiers...>::type;

    template<class ElementType, class Extents, class LayoutPolicy,
            class AccessorPolicy, class... SliceSpecifiers>
    mdspan_subspan_t<ElementType, Extents, LayoutPolicy,
                    AccessorPolicy, SliceSpecifiers...>
        subspan(const basic_mdspan<ElementType, Extents, LayoutPolicy,
                                AccessorPolicy>& src, SliceSpecifiers ... slices) noexcept;

    // tag supporting subspan
    struct all_type { explicit all_type() = default; };
    inline constexpr all_type all = all_type{};
}}}

```

26.7.❖ Class template `extents` [mdspan.extents]

26.7.❖.1 Overview [mdspan.extents.overview]

1. A *multidimensional index space* is the Cartesian product $[0, N_0) \times [0, N_1) \times \dots \times [0, N_{R-1})$ of half-open integer intervals.
2. The *rank* of a multidimensional index space is the number of intervals R .
3. An `extents` object `e` defines a multidimensional index space where R is `e.rank()` and N_r is `e.extent(r)`.
4. A *multidimensional index* is an element of a multidimensional index space. We may identify a pack of integer type `i...` with a multidimensional index in the multidimensional index space defined by an `extents` object `e`, as long as:
 - `sizeof...(i) == e.rank()` is true; and
 - for all `r` in the range $[0, e.rank())$, the `r`-th element of `i...` is in the range $[0, e.extent(r))$.

```
namespace std {
namespace experimental {
inline namespace fundamentals_v3 {

template<ptrdiff_t... StaticExtents>
class extents {
public:
    // types
    using index_type = ptrdiff_t;

    // [mdspan.extents.cons], Constructors and assignment
    constexpr extents() noexcept;
    constexpr extents(const extents&) noexcept;
    template<class... IndexType>
    constexpr explicit extents(IndexType... dynamic);
    template<class IndexType>
    constexpr extents(const array<IndexType, rank_dynamic()>& dynamic) noexcept;
    template<ptrdiff_t... OtherExtents>
    constexpr extents(const extents<OtherExtents...>& other);
    ~extents() = default;

    constexpr extents& operator=(const extents&) noexcept = default;
    template<ptrdiff_t... OtherExtents>
    constexpr extents& operator=(const extents<OtherExtents...>& other);

    // [mdspan.extents.obs], Observers of the domain multidimensional index space
    static constexpr size_t rank() noexcept;
    static constexpr size_t rank_dynamic() noexcept;
    static constexpr index_type static_extent(size_t) noexcept;
    constexpr index_type extent(size_t) const noexcept;

private:
    array<index_type, (size_t(StaticExtents == dynamic_extent) + ...) > dynamic_extents_; // exposition only
    static size_t dynamic_rank(size_t); // exposition only
};

}}}
```

5. If `((StaticExtents >= 0) || (StaticExtents == dynamic_extent) && ...)` is false, then the program is ill-formed.
6. Each element of the `StaticExtents` pack equal to `dynamic_extent` denotes a *dynamic extent*.
7. The exposition only `dynamic_extents_` member contains an element for each dynamic extent.
8. Let `StaticExtent[r]` denote the `r`-th element of the `StaticExtents` pack. The exposition only `dynamic_rank` member function is defined as

```
static size_t dynamic_rank(size_t i) // exposition only
{
    size_t j = 0;
    for (size_t r = 0; r < i; ++r)
        if (StaticExtents[r] == dynamic_extent)
            ++j;
    return j;
}
```

26.7.2 Constructors and assignment [mdspan.extents.cons]

```
constexpr extents() noexcept;
```

- *Effects:* Value-initializes `dynamic_extents_` to `{ }`.

```
constexpr extents(const extents& other) noexcept;
```

- *Effects:* Initializes `dynamic_extents_` with `other.dynamic_extents_`.

```
template<ptrdiff_t... OtherExtents>
constexpr extents(const extents<OtherExtents...>& other);
```

- *Constraints:*
 - `sizeof...(StaticExtents)` equals `sizeof...(OtherExtents)`; and
 - For all `r` in the range `[0, rank())` where `static_extent(r) != dynamic_extent` and `other.static_extent(r) != dynamic_extent` are both true, `static_extent(r) == other.static_extent(r)` is true.
- *Expects:* For all `r` in the range `[0, rank())`, `static_extent(r) == dynamic_extent || static_extent(r) == other.extent(r)` is true.
- *Effects:* For each `r` in the range `[0, rank())` where `static_extent(r) == dynamic_extent` is true, initializes `dynamic_extent[dynamic_rank(r)]` with `other.extent(r)`.
- *Throws:* Nothing.

```
template<class... IndexType>
constexpr explicit extents(IndexType... dynamic);
```

- *Constraints:*
 - `(is_convertible_v<IndexType, index_type> && ...) is true`, and
 - `sizeof...(dynamic) == rank_dynamic()` is true. *[Note: If `rank_dynamic()` is zero, overload resolution will unambiguously pick the default constructor of `extents` over this constructor. — end note]*
- *Expects:* `((dynamic_extents >= 0) && ...)` is true.
- *Effects:* Initializes `dynamic_extents_` with `dynamic...`.
- *Throws:* Nothing.

```
template<class IndexType>
constexpr extents(const array<IndexType, rank_dynamic()>& dynamic) noexcept;
```

- *Constraints:* `is_convertible_v<IndexType, index_type>` is true.
- *Expects:* `dynamic[i] >= 0` for all `i` in the range `[0, rank_dynamic())`.
- *Effects:* Initializes `dynamic_extents_` with `dynamic`.

```
template<ptrdiff_t... OtherExtents>
constexpr extents& operator=(const extents<OtherExtents...>& other);
```

- *Constraints:*
 - `sizeof...(StaticExtents)` equals `sizeof...(OtherExtents)`; and
 - For all `r` in the range `[0, rank())` where `static_extent(r) != dynamic_extent` and `other.static_extent(r) != dynamic_extent` are both true, `static_extent(r) == other.static_extent(r)` is true.
- *Expects:* For all `r` in the range `[0, rank())`, `static_extent(r) == dynamic_extent || static_extent(r) == other.extent(r)` is true.
- *Effects:* For each `r` in the range `[0, rank())` where `static_extent(r) == dynamic_extent` is true, assigns `other.extent(r)` to `dynamic_extent[dynamic_rank(r)]`.

26.7.4.3 Observers of the domain multidimensional index space [mdspan.extents.obs]

```
static constexpr size_t rank() noexcept;
```

- *Returns:* `sizeof...(StaticExtents)`.

```
static constexpr size_t rank_dynamic() noexcept;
```

- *Returns:*
 - If `(sizeof...(StaticExtents) == 0)` is true, then zero;
 - Otherwise, the number of elements of `StaticExtents...` that equal `dynamic_extent`.

```
static constexpr index_type static_extent(size_t r) noexcept;
```

- *Returns:*
 - If `r` is in the range `[0, rank())`, then the `r`-th entry in the `StaticExtents` parameter pack;
 - Otherwise, 1.

```
constexpr index_type extent(size_t r) const noexcept;
```

- *Returns:*
 - If `static_extent(r) == dynamic_extent` is true, then `dynamic_extents_[dynamic_rank(r)]`;
 - Otherwise, `static_extent(r)`.

26.7.4.4 `extents` comparison operators [mdspan.extents.compare]

```
template<ptrdiff_t... LHS, ptrdiff_t... RHS>
constexpr bool operator==(const extents<LHS...>& lhs, const extents<RHS...>& rhs) noexcept
t;
```

- **Returns:**

- If `lhs.rank() == rhs.rank()` is true, and if `lhs.extent(r) == rhs.extent(r)` is true for all `r` in the range `[0, lhs.rank())`, then true;
- Otherwise, false.

```
template<ptrdiff_t... LHS, ptrdiff_t... RHS>
constexpr bool operator!=(const extents<LHS...>& lhs, const extents<RHS...>& rhs) noexcept
t;
```

- **Effects:** Equivalent to: `return !(lhs == rhs).`

26.7. ♦ Layout mapping policy [mdspan.layout]

26.7. ♦.1 Layout mapping requirements [mdspan.layout.reqs]

1. A *layout mapping policy* is a class that contains a *layout mapping*, which is a nested class template.
2. A layout mapping policy and its layout mapping nested class template meet the requirements in Table ♦.
3. A layout mapping meets the requirements of *Cpp17DefaultConstructible*, *Cpp17CopyAssignable*, *Cpp17EqualityComparable*, and the requirements in Table ♦.
4. In Table ♦:
 - `MP` denotes a layout mapping policy.
 - `M` denotes a specialization of the layout mapping policy's nested layout mapping template class.
 - `E` denotes a specialization of `extents`.
 - `e` denotes an object of type `E`.
 - `m` denotes an object of type `M`.
 - `i...` and `j...` are multidimensional indices in the multidimensional index space defined by `e`.
 - `r` is an integral value in the range `[0, e.rank())`.

Table ♦ — Layout mapping policy and layout mapping requirements

Expression	Return Type	Returns	Requires
<code>MP::template mapping<E></code>	<code>M</code>		
<code>m.extents()</code>	<code>E</code>	Returns: <code>e</code> .	
<code>m(i...)</code>	<code>E::index_type</code>	Returns: A nonnegative value, the layout mapping of <code>i...</code> .	
<code>m.required_span_size()</code>	<code>E::index_type</code>	Returns: If <code>e.extent(r)</code> is zero for any <code>r</code> in the range <code>[0, e.rank())</code> , then zero, else 1 plus the maximum value of <code>m(i...)</code> for all <code>i...</code> in <code>e</code> .	
<code>m.is_unique()</code>	<code>bool</code>	Returns: true if for every <code>i... != j...</code> , <code>m(i...) != m(j...)</code> is true.	
<code>m.is_contiguous()</code>	<code>bool</code>	Returns: true if the set defined by <code>m(i)...</code> equals the set <code>{0, ..., m.required_span_size() - 1}</code> .	
<code>m.is_strided()</code>	<code>bool</code>	Returns: true if, for all <code>i</code> in <code>[0,</code>	

		$e.rank()$, there exists an integer r , such that for all $j...$ and $i...$ in e , if all elements of $j...$ and $i...$ are equal except for the r -th element, where r equals $r + 1$, then r equals $m(j...) - m(i...)$.	
<code>M::is_always_unique()</code>	<code>bool</code>	Returns: true if <code>m.is_unique()</code> is true for any object of type <code>M</code> .	
<code>M::is_always_contiguous()</code>	<code>bool</code>	Returns: true if <code>m.is_contiguous()</code> is true for any object of type <code>M</code> .	
<code>M::is_always_strided()</code>	<code>bool</code>	Returns: true if <code>m.is_strided()</code> is true for any object of type <code>M</code> .	
<code>m.stride(r)</code>	<code>E::index_type</code>	Returns: The integer r , such that for all $j...$ and $i...$ in e , if all elements of $j...$ and $i...$ are equal except for the r -th element, where r equals $r + 1$, then r equals $m(j...) - m(i...)$. [Note: r has the same definition here as in <code>m.is_strided()</code> above. —end note] We call r the <i>stride of ordinate r</i> .	Requires: <code>m.is_strided()</code> is true.

26.7.2 Class template `layout_left` [`mdspan.layout.left`]

- `layout_left` meets the requirements of layout mapping policy.
- `layout_left` gives a layout mapping where the left-most extent is stride one and strides increase left-to-right as the product of extents.
- If `Extents` is not a (possibly cv-qualified) specialization of `extents`, then the program is ill-formed.

```

namespace std {
namespace experimental {
inline namespace fundamentals_v3 {

struct layout_left {
    template<class Extents>
    class mapping {
    public:
        // [mdspan.layout.left.cons], layout_left::mapping constructors
        constexpr mapping() noexcept;
        constexpr mapping(const mapping& other) noexcept;
        constexpr mapping(mapping&& other) noexcept;
        constexpr mapping(const Extents& e) noexcept;
        template<class OtherExtents>
            constexpr mapping(const mapping<OtherExtents>& other);

        mapping& operator=() noexcept = default;
        mapping& operator=(const mapping& other) noexcept = default;
        template<class OtherExtents>
            constexpr mapping& operator=(const mapping<OtherExtents>& other);

        // [mdspan.layout.left.ops], layout_left::mapping operations
        Extents extents() const noexcept;

        constexpr typename Extents::index_type required_span_size() const noexcept;

        template<class... Indices>
            typename Extents::index_type operator()(Indices... is) const;

        static constexpr bool is_always_unique();
        static constexpr bool is_always_contiguous();
        static constexpr bool is_always_strided();

        constexpr bool is_unique() const;
        constexpr bool is_contiguous() const;
        constexpr bool is_strided() const;

        template<class OtherExtents>
            constexpr bool operator==(const mapping<OtherExtents>& other) const noexcept;
        template<class OtherExtents>
            constexpr bool operator!=(const mapping<OtherExtents>& other) const noexcept;

        typename Extents::index_type stride(size_t rank) const noexcept;

    private:
        Extents extents_; // exposition only
    };
};

}}}

```

26.7.2.1 `layout_left::mapping` constructors [mdspan.layout.left.cons]

```
constexpr mapping() noexcept;
```

- *Effects:* Default-initializes `extents_`.
- *Ensures:* `extents() == Extents()` is true.

```
constexpr mapping(const mapping& other) noexcept;
```

- *Effects*: Initializes `extents_` with `other.extents_`.
- *Ensures*: `extents() == other.extents()` is true.

```
constexpr mapping(mapping&& other) noexcept;
```

- *Effects*: Initializes `extents_` with `move(other.extents_)`.
- *Ensures*: `extents()` equals the result of `other.extents()` before the invocation of the move.

```
constexpr mapping(const Extents& e) noexcept;
```

- *Effects*: Initializes `extents_` with `e`.
- *Ensures*: `extents() == e` is true.

```
template<class OtherExtents>
constexpr mapping(const mapping<OtherExtents>& other);
```

- *Requires*: `other.extents()` meets the requirements for use in the initialization of `extents_`.
- *Effects*: Initializes `extents_` with `other.extents()`.
- *Ensures*: `extents() == other.extents()` is true.
- *Throws*: Nothing.

26.7.2.2 `layout_left::mapping` operations [mdspan.layout.left.ops]

```
Extents extents() const noexcept;
```

- *Returns*: `extents_`.

```
typename Extents::index_type required_span_size() const noexcept;
```

- *Returns*: The product of `extents().extent(r)` for all `r` in the range `[0, extents().rank())`.

```
template<class... Indices>
typename Extents::index_type operator()(Indices... i) const;
```

- *Constraints*:
 - `sizeof...(Indices) == extents().rank()` is true, and
 - `(is_convertible_v<Indices, typename Extents::index_type> && ...)` is true.
- *Requires*: `0 <= array{i...}[r] < extents().extent(r)` for all `r` in the range `[0, extents().rank())`.

- *Returns:* Let k denote the k -th element of `i...`, and let `equal` equal `extents().rank()`:
 - If `equal` is zero, then zero;
 - Otherwise, the sum of $k \times \text{stride}(k)$ for all k in the range `[0, equal)`.

```
static constexpr bool is_always_unique();
static constexpr bool is_always_contiguous();
static constexpr bool is_always_strided();
constexpr bool is_unique() const;
constexpr bool is_contiguous() const;
constexpr bool is_strided() const;
```

- *Returns:* `true`.

```
typename Extents::index_type stride(size_t r) const;
```

- *Returns:*
 - If `r` is zero, then one;
 - Otherwise, the product of `extents().extent(k)` for all k in the range `[0, r)`.

```
template<class OtherExtents>
constexpr bool operator==(const mapping<OtherExtents>& other) const noexcept;
```

- *Effects:* Equivalent to: `return extents() == other.extents();`.

```
template<class OtherExtents>
constexpr bool operator!=(const mapping<OtherExtents>& other) const noexcept;
```

- *Effects:* Equivalent to: `return extents() != other.extents();`.

26.7.3 Class template `layout_right` [mdspan.layout.right]

1. `layout_right` meets the requirements of layout mapping policy.
2. The layout mapping property `layout_right` gives a layout mapping where the right-most extent is stride one and strides increase right-to-left as the product of extents.
3. If `Extents` is not a (possibly cv-qualified) specialization of `extents`, then the program is ill-formed.


```

namespace std {
namespace experimental {
inline namespace fundamentals_v3 {

struct layout_right {
    template<class Extents>
    class mapping {
    public:
        // [mdspan.layout.right.cons], layout_left::mapping constructors
        constexpr mapping() noexcept;
        constexpr mapping(const mapping& other) noexcept;
        constexpr mapping(mapping&& other) noexcept;
        constexpr mapping(const Extents& e) noexcept;
        template<class OtherExtents>
            constexpr mapping(const mapping<OtherExtents>& other);

        mapping& operator=() noexcept = default;
        mapping& operator=(const mapping& other) noexcept = default;
        template<class OtherExtents>
            constexpr mapping& operator=(const mapping<OtherExtents>& other);

        // [mdspan.layout.right.ops], layout_right::mapping operations
        Extents extents() const noexcept;

        constexpr typename Extents::index_type required_span_size() const noexcept;

        template<class... Indices>
            typename Extents::index_type operator()(Indices... is) const;

        static constexpr bool is_always_unique() noexcept;
        static constexpr bool is_always_contiguous() noexcept;
        static constexpr bool is_always_strided() noexcept;

        constexpr bool is_unique() const noexcept;
        constexpr bool is_contiguous() const noexcept;
        constexpr bool is_strided() const noexcept;

        typename Extents::index_type stride(size_t rank) const noexcept;

        template<class OtherExtents>
            constexpr bool operator==(const mapping<OtherExtents>& other) const noexcept;
        template<class OtherExtents>
            constexpr bool operator!=(const mapping<OtherExtents>& other) const noexcept;

    private:
        Extents extents_; // exposition only
    };
};
}

```

26.7.3.1 `layout_right::mapping` constructors [mdspan.layout.right.cons]

```
constexpr mapping() noexcept;
```

- *Effects:* Default-initializes `extents_`.
- *Ensures:* `extents() == Extents()` is true.

```
constexpr mapping(const mapping& other) noexcept;
```

- *Effects*: Initializes `extents_` with `other.extents_`.
- *Ensures*: `extents() == other.extents()` is true.

```
constexpr mapping(mapping&& other) noexcept;
```

- *Effects*: Initializes `extents_` with `move(other.extents_)`.
- *Ensures*: `extents()` equals the result of `other.extents()` before the invocation of the move.

```
constexpr mapping(const Extents& e) noexcept;
```

- *Effects*: Initializes `extents_` with `e`.
- *Ensures*: `extents() == e` is true.

```
template<class OtherExtents>
constexpr mapping(const mapping<OtherExtents>& other);
```

- *Requires*: `other.extents()` meets the requirements for use in the initialization of `extents_`.
- *Effects*: Initializes `extents_` with `other.extents()`.
- *Ensures*: `extents() == other.extents()` is true.
- *Throws*: Nothing.

26.7.3.2 `layout_right::mapping` operations [mdspan.layout.right.ops]

```
Extents extents() const noexcept;
```

- *Returns*: `extents_`.

```
typename Extents::index_type required_span_size() const noexcept;
```

- *Returns*: The product of `extents().extent(r)` for all `r` in the range `[0, extents().rank())`.

```
template<class... Indices>
typename Extents::index_type operator()(Indices... i) const noexcept;
```

- *Constraints*:
 - `sizeof...(Indices) == extents().rank()` is true, and
 - `(is_convertible_v<Indices, typename Extents::index_type> && ...)` is true.
- *Requires*: `0 <= array{i...}[r] < extents().extent(r)` for all `r` in the range `[0, extents().rank())`.
- *Returns*: Let k denote the k -th element of `i...`, and let `equal` equal `extents().rank()`:

- If `is` is zero, then zero;
- Otherwise, the sum of `k × stride(k)` for all `k` in the range `[0, )`.

```
static constexpr bool is_always_unique() noexcept;
static constexpr bool is_always_contiguous() noexcept;
static constexpr bool is_always_strided() noexcept;
constexpr bool is_unique() const noexcept;
constexpr bool is_contiguous() const noexcept;
constexpr bool is_strided() const noexcept;
```

- *Returns:* `true`.

```
typename Extents::index_type stride(size_t r) const noexcept;
```

- *Effects:* Equivalent to:

```
Extents::index_type s = 1;
for(size_t k = r+1; k < extents().rank(); ++k)
    s *= extents().extent(k);
return s;
```

```
template<class OtherExtents>
constexpr bool operator==(const mapping<OtherExtents>& other) const noexcept;
```

- *Effects:* Equivalent to: `return extents() == other.extents();`.

```
template<class OtherExtents>
constexpr bool operator!=(const mapping<OtherExtents>& other) const noexcept;
```

- *Effects:* Equivalent to: `return extents() != other.extents();`.

26.7.4 Class template `layout_stride` [mdspan.layout.stride]

1. `layout_stride` meets the requirements of layout mapping policy.
2. The layout mapping property `layout_stride` gives a layout mapping where the strides are user defined.
3. If `Extents` is not a (possibly cv-qualified) specialization of `extents`, then the program is ill-formed.

```

namespace std {
namespace experimental {
inline namespace fundamentals_v3 {

struct layout_stride {
    template<class Extents>
    class mapping {
    public:
        // [mdspan.layout.stride.cons], layout_stride::mapping constructors
        constexpr mapping() noexcept;
        constexpr mapping(mapping const& other) noexcept;
        constexpr mapping(mapping&& other) noexcept;
        constexpr mapping(const Extents& e, const array<typename Extents::index_type, Extents::
rank()>& s) noexcept;
        template<class OtherExtents>
            constexpr mapping(const mapping<OtherExtents>& other);

        mapping& operator=() noexcept = default;
        mapping& operator=(const mapping& other) noexcept = default;
        template<class OtherExtents>
            constexpr mapping& operator=(const mapping<OtherExtents>& other);

        // [mdspan.layout.stride.ops], layout_stride::mapping operations
        Extents extents() const noexcept;
        array<typename Extents::index_type, Extents::rank()> strides() const noexcept;

        constexpr typename Extents::index_type required_span_size() const noexcept;

        template<class... Indices>
            typename Extents::index_type operator()(Indices... is) const;

        static constexpr bool is_always_unique() noexcept;
        static constexpr bool is_always_contiguous() noexcept;
        static constexpr bool is_always_strided() noexcept;

        constexpr bool is_unique() const noexcept;
        constexpr bool is_contiguous() const noexcept;
        constexpr bool is_strided() const noexcept;

        typename Extents::index_type stride(size_t rank) const noexcept;

        template<class OtherExtents>
            constexpr bool operator==(const mapping<OtherExtents>& other) const noexcept;
        template<class OtherExtents>
            constexpr bool operator!=(const mapping<OtherExtents>& other) const noexcept;

    private:
        Extents extents_; // exposition only
        array<typename Extents::index_type, Extents::rank()> strides_; // exposition only
    };
};
}}}

```

26.7.4.1 layout_stride::mapping constructors [mdspan.layout.stride.cons]

```
constexpr mapping() noexcept;
```

- *Effects:* Default-initializes `extents_`.
- *Ensures:*
 - `extents() == Extents()` is true, and
 - `strides() == array<typename Extents::index_type, Extents::rank()>()` is true.

```
constexpr mapping(const mapping& other) noexcept;
```

- *Effects*: Initializes `extents_` with `other.extents_`.
- *Ensures*:
 - `extents() == other.extents()` is true, and
 - `strides() == other.strides()` is true.

```
constexpr mapping(mapping&& other) noexcept;
```

- *Effects*: Initializes `extents_` with `move(other.extents_)`.
- *Ensures*:
 - `extents()` equals the result of `other.extents()` before the invocation of the move, and
 - `strides()` equals the result of `other.strides()` before the invocation of the move.

```
constexpr mapping(const Extents& e, array<typename Extents::index_type, Extents::rank()> s)
noexcept;
```

Let `e` equal `Extents::rank()`. If `p` is a permutation of the integers 0, ..., `e-1`, then let `p` be an exposition-only array of `size_t` of length `e` such that `p[i]` equals `i` for all `i` in the range `[0, e)`.

- *Requires*:
 - `s[i] > 0` is true for all `i` in the range `[0, e)`.
 - If `R` is nonzero, then there exists a permutation `q` of the integers 0, ..., `e-1`, such that `s(p[i]) >= s(p[i-1]) * e.extent(p[i-1])` is true for `i` in the range `[1, e)`.
- *Effects*:
 - Initializes `extents_` with `e`, and
 - initializes `strides_` with `s`.
- *Ensures*:
 - `extents() == e` is true, and
 - `strides() == s` is true.
- *Throws*: Nothing.

```
template<class OtherExtents>
constexpr mapping(const mapping<OtherExtents>& other);
```

- *Requires*: `other.extents()` meets the requirements for use in the initialization of `extents_`.
- *Effects*:
 - Initializes `extents_` with `other.extents()`, and
 - initializes `strides_` with `other.strides()`.
- *Ensures*:
 - `extents() == other.extents()` is true, and
 - `strides() == other.strides()` is true.
- *Throws*: Nothing.

26.7.4.2 layout_stride::mapping operations [mdspan.layout.stride.ops]

```
Extents extents() const noexcept;
```

- Returns: `extents_`.

```
array<typename Extents::index_type, Extents::rank()> strides() const noexcept;
```

- Returns: `strides_`.

```
typename Extents::index_type required_span_size() const noexcept;
```

- Returns: The maximum of `extents().extent(r) * stride(r)` for all `r` in the range `[0, extents().rank())`.

```
template <class... Indices>
    typename Extents::index_type operator()(Indices... i) const;
```

- Constraints:
 - `sizeof...(Indices) == Extents::rank()` is true, and
 - `(is_convertible_v<Indices, typename Extents::index_type> && ...)` is true.
- Requires: `0 <= array{i...}[r] < extents().extent(r)` for all `r` in the range `[0, extents().rank())`.
- Returns: Let k denote the k -th element of `i...`, let `rank` equal `Extents::rank()`, and let `s` be `strides()`:
 - If `rank` is zero, then zero;
 - Otherwise, the sum of $k \times s[k]$ for all k in the range `[0, rank)`.
- Throws: Nothing.

```
static constexpr bool is_always_unique() noexcept;
static constexpr bool is_always_strided() noexcept;
constexpr bool is_unique() const noexcept;
constexpr bool is_strided() const noexcept;
```

- Returns: `true`.

```
static constexpr bool is_always_contiguous() noexcept;
```

- Returns: `false`.

```
constexpr bool is_contiguous() const noexcept;
```

Let `R` equal `Extents::rank()`. If `p` is a permutation of the integers `0, ..., R-1`, then let `p` be an exposition-only array of `size_t` of length `R` such that `p[i]` equals `i` for all `i` in the range `[0, R)`.

- Returns:
 - If `R` is zero, then `true`;
 - Else, if there is a permutation `q` of the integers `0, ..., R-1` such that `min(stride(p[i]))` equals `1` for `i` in the range `[0, R)`, and `stride(p[i])` equals `stride(p[i-1]) * extents().extent(p[i-1])` for `i` in the range `[1, R)`, then `true`;
 - Otherwise, `false`.

```
typename Extents::index_type stride(size_t r) const noexcept;
```

- Returns: `strides_[r]`.

```
template<class OtherExtents>
constexpr bool operator==(const mapping<OtherExtents>& other) const noexcept;
```

- *Effects*: Equivalent to: `return extents() == other.extents();`.

```
template<class OtherExtents>
constexpr bool operator!=(const mapping<OtherExtents>& other) const noexcept;
```

- *Effects*: Equivalent to: `return extents() != other.extents();`.

26.7.◆ Accessor Policy [mdspan.accessor]

An *accessor policy* defines types and operations by which a contiguous set of objects of a particular type are accessed.

26.7.◆.1 Accessor policy requirements [mdspan.accessor.reqs]

An accessor policy defines:

- a handle to a single element of type `element_type`;
- a handle to a contiguous set of elements of type `element_type`, accessible through the policy's `access` function;
- conversion of a handle to a contiguous set of elements, to a pointer **[conv.array]**; and
- getting a handle to the contiguous subset of elements beginning at an integer offset value.

[*Note*: The type of `reference` need not be `element_type&`. The type of `pointer` need not be `element_type*`. — *end note*]

[*Note*: The constructor of the object representing the contiguous set of objects may impose additional restrictions on the contiguous set of objects, such as restricting `element_type` to be trivially copyable or requiring a larger alignment than `alignment_of_v<element_type>`. — *end note*]

In Table ◆:

- `A` denotes an accessor policy.
- `a` denotes an object of type `A`.
- `p` denotes an object of type `A::pointer`.
- `i` and `j` each denote a `ptrdiff_t` value.

Table ◆: Accessor policy requirements

Expression	Return Type	Notes/Returns/Requires
<code>A::element_type</code>		Type of each element in the contiguous set of elements.
<code>A::pointer</code>		Type through which the range of elements are accessed. <i>Requires</i> : <code>Cpp17DefaultConstructible</code> , <code>Cpp17CopyConstructible</code> , and <code>Cpp17CopyAssignable</code> .
<code>A::reference</code>		Type through which an element is accessed.
<code>A::offset_policy</code>		Accessor policy for accessing a pointer returned by <code>a.offset(p, i)</code> . <i>Requires</i> :

		• <code>A::offset_policy</code> meets the requirements of an accessor policy in Table ◆. • <code>is_convertible_v<A, A::offset_policy></code> is true and <code>A::offset_policy(a)</code> is valid.
<code>a.decay(p)</code>	<code>A::element_type*</code>	<i>Returns:</i> A pointer that references the same location as <code>p</code> .
<code>a.access(p, i)</code>	<code>A::reference</code>	<i>Returns:</i> An object which provides access to the <code>i</code> -th element in the range of elements that starts at <code>p</code> .
<code>a.offset(p, i)</code>	<code>A::offset_policy::pointer</code>	<i>Returns:</i> A pointer that references the same location as <code>a.decay(p)+i</code> . <i>Requires:</i> • <code>a.decay(p)+i</code> equals <code>A::offset_policy(a).decay(a.offset(p, i))</code> • <code>A::offset_policy(a).access(a.offset(p, i), j)</code> is valid if <code>a.access(p, i+j)</code> is valid.

26.7.◆.2 Class template `accessor_basic` [`mdspan.accessor.basic`]

1. `accessor_basic` meets the requirements of accessor policy.
2. `ElementType` is required to be a complete object type that is neither an abstract class type nor an array type.

```
namespace std {
namespace experimental {
inline namespace fundamentals_v3 {

    template<class ElementType>
    struct accessor_basic {
        using offset_policy = accessor_basic;
        using element_type = ElementType;
        using reference = ElementType&;
        using pointer = ElementType*;

        constexpr typename offset_policy::pointer
            offset(pointer p, ptrdiff_t i) const noexcept;

        constexpr reference access(pointer p, ptrdiff_t i) const noexcept;

        constexpr pointer decay(pointer p) const noexcept;
    };

}}}
```

```
constexpr typename offset_policy::pointer
    offset(pointer p, ptrdiff_t i) const noexcept;
```

- *Requires:* `p + i` is dereferenceable.
- *Returns:* `p + i`.

```
constexpr reference access(pointer p, ptrdiff_t i) const noexcept;
```

- *Requires:* `p + i` is dereferenceable.
- *Returns:* `p[i]`.


```
constexpr pointer decay(pointer p) const noexcept;
```

- Returns: `p`.

26.7.❖ Class template `basic_mdspan` [mdspan.basic]

26.7.❖.1 `basic_mdspan` overview [mdspan.basic.overview]

1. `basic_mdspan` maps a multidimensional index in its domain to a reference to an element in its codomain `span`.
2. The *domain* of a `basic_mdspan` object is a multidimensional index space, defined by $\text{extent}(0) \times [0, \text{extent}(1)) \times \dots \times [0, \text{extent}(\text{rank}() - 1))$. Each extent may be statically or dynamically specified.
3. The *codomain* of a `basic_mdspan` object is a `span` of elements.
4. As with `span`, the storage of the objects in the codomain `span` of a `basic_mdspan` is owned by some other object.

```
namespace std {
namespace experimental {
inline namespace fundamentals_v3 {

template<class ElementType, class Extents, class LayoutPolicy, class AccessorPolicy>
class basic_mdspan {
public:

    // Domain and codomain types
    using extents_type = Extents;
    using layout_type = LayoutPolicy;
    using accessor_type = AccessorPolicy;
    using mapping_type = typename layout_type::template mapping_type<extents_type>;
    using element_type = typename accessor_type::element_type;
    using value_type = remove_cv_t<element_type>;
    using index_type = ptrdiff_t;
    using difference_type = ptrdiff_t;
    using pointer = typename accessor_type::pointer;
    using reference = typename accessor_type::reference;

    // [mdspan.basic.cons], basic_mdspan constructors, assignment, and destructor
    constexpr basic_mdspan() noexcept = default;
    constexpr basic_mdspan(const basic_mdspan&) noexcept = default;
    constexpr basic_mdspan(basic_mdspan&&) noexcept;
    template<class... IndexType>
        explicit constexpr basic_mdspan(pointer p, IndexType... dynamic_extents);
    template<class IndexType, size_t N>
        explicit constexpr basic_mdspan(pointer p, const array<IndexType, N>& dynamic_extents);
    constexpr basic_mdspan(pointer p, const mapping_type& m);
    constexpr basic_mdspan(pointer p, const mapping_type& m, const accessor_type& a);
    template<class OtherElementType, class OtherExtents, class OtherLayoutPolicy, class Other
    AccessorPolicy>
        constexpr basic_mdspan(const basic_mdspan<OtherElementType, OtherExtents, OtherLayoutPo
    licy, OtherAccessorPolicy>& other);

    ~basic_mdspan() = default;

    constexpr basic_mdspan& operator=(const basic_mdspan&) noexcept = default;
    constexpr basic_mdspan& operator=(basic_mdspan&&) noexcept = default;
    template<class OtherElementType, class OtherExtents, class OtherLayoutPolicy, class Other
    AccessorPolicy>
        constexpr basic_mdspan& operator=(const basic_mdspan<OtherElementType, OtherExtents, Ot
    herLayoutPolicy, OtherAccessorPolicy>& other) noexcept;
```

```

// [mdspan.basic.mapping], basic_mdspan mapping domain multidimensional index to access c
odomain element
constexpr reference operator[](index_type) const noexcept;
template<class... IndexType>
    constexpr reference operator()(IndexType... indices) const noexcept;
template<class IndexType, size_t N>
    constexpr reference operator()(const array<IndexType, N>& indices) const noexcept;
accessor_type accessor() const;

// [mdspan.basic.domobs], basic_mdspan observers of the domain multidimensional index spa
ce
static constexpr int rank() noexcept;
static constexpr int rank_dynamic() noexcept;
static constexpr index_type static_extent(size_t) noexcept;

constexpr Extents extents() const noexcept;
constexpr index_type extent(size_t) const noexcept;
constexpr index_type size() const noexcept;
constexpr index_type unique_size() const noexcept;

// [mdspan.basic.codomain], basic_mdspan observers of the codomain
constexpr span<element_type> span() const noexcept;
constexpr pointer data() const noexcept;

// [mdspan.basic.obs], basic_mdspan observers of the mapping
static constexpr bool is_always_unique() noexcept;
static constexpr bool is_always_contiguous() noexcept;
static constexpr bool is_always_strided() noexcept;

constexpr mapping_type mapping() const noexcept;
constexpr bool is_unique() const noexcept;
constexpr bool is_contiguous() const noexcept;
constexpr bool is_strided() const noexcept;
constexpr index_type stride(size_t) const;

private:
    accessor_type acc_; // exposition only
    mapping_type map_; // exposition only
    pointer ptr_; // exposition only
};
}}}

```

5. `ElementType` is required to be a complete object type that is neither an abstract class type nor an array type.
6. If `Extents` is not a (cv-unqualified) specialization of `extents`, then the program is ill-formed.
7. If `LayoutPolicy` does not meet the layout mapping policy requirements, then the program is ill-formed.
8. If `AccessorPolicy` does not meet the accessor policy requirements or if `std::is_same_v<typename AccessorPolicy::element_type, ElementType>` is false, then the program is ill-formed.

26.7.❖.1 `basic_mdspan` constructors, assignment, and destructor [mdspan.basic.cons] // Mapping domain multidimensional index to access codomain element

```
constexpr basic_mdspan() noexcept = default;
```

- *Effects:*
 - Zero-initializes `ptr_`,
 - Value-initializes `map_`, and
 - Value-initializes `acc_`.

- *Ensures:*
 - `size() == 0` is true,
 - `extents() == Extents()` is true, and
 - `mapping() == mapping_type()` is true.

```
constexpr basic_mdspan(const basic_mdspan& other) noexcept = default;
```

- *Effects:*
 - Initializes `ptr_` with `other.ptr_`,
 - Initializes `map_` with `other.map_`, and
 - Initializes `acc_` with `other.acc_`.
- *Ensures:*
 - `size() == other.size()` is true,
 - `extents() == other.extents()` is true, and
 - `mapping() == other.mapping()` is true.

```
constexpr basic_mdspan(basic_mdspan&& other) noexcept;
```

- *Effects:*
 - Initializes `ptr_` with `move(other.ptr_)`,
 - Initializes `map_` with `move(other.map_)`, and
 - Initializes `acc_` with `move(other.acc_)`.

```
template<class... IndexType>
explicit constexpr basic_mdspan(pointer ptr, IndexType... dynamic_extents);
```

- *Constraints:*
 - `(is_convertible_v<IndexType, index_type> && ...)` is true,
 - `sizeof...(dynamic_extents) == rank_dynamic()` is true,
 - `is_constructible_v<mapping_type, Extents>` is true, and
 - `is_default_constructible_v<accessor_type>` is true.
- *Effects:*
 - Initializes `ptr_` with `ptr`,
 - Initializes `map_` with `Extents(dynamic_extents...)`, and
 - Value-initializes `acc_`.
- *Ensures:*
 - `extents() == Extents(dynamic_extents...)` is true, and
 - `mapping() == mapping_type(Extents(dynamic_extents...))` is true.
- *Throws:* Nothing.

```
template<class IndexType, size_t N>
    explicit constexpr basic_mdspan(pointer p, const array<IndexType, N>& dynamic_extents);
```

- *Constraints:*
 - `is_convertible_v<IndexType, index_type>` is true,
 - `N == rank_dynamic()` is true,
 - `is_constructible_v<mapping_type, Extents>` is true, and
 - `is_default_constructible_v<accessor_type>` is true.
- *Effects:* Equivalent to: `basic_mdspan(p, dynamic_extents[Rs]...)`, with `Rs...` from `index_sequence<Rs...>` matching `make_index_sequence<N>`.
- *Throws:* Nothing.

```
constexpr basic_mdspan(pointer p, const mapping_type& m);
```

- *Constraints:* `is_default_constructible_v<accessor_type>` is true.
- *Effects:*
 - Initializes `ptr_` with `p`,
 - Initializes `map_` with `m`, and
 - Value-initializes `acc_`.
- *Ensures:*
 - `extents() == m.extents()` is true, and
 - `mapping() == m` is true.
- *Throws:* Nothing.

```
constexpr basic_mdspan(pointer p, const mapping_type& m, const accessor_type& a);
```

- *Effects:*
 - Initializes `ptr_` with `p`,
 - Initializes `map_` with `m`, and
 - Initializes `acc_` with `a`.
- *Ensures:*
 - `extents() == m.extents()` is true, and
 - `mapping() == m` is true.
- *Throws:* Nothing.

```
template<class OtherElementType, class OtherExtents, class OtherLayoutPolicy, class OtherAccessor>
    constexpr basic_mdspan(const basic_mdspan<OtherElementType, OtherExtents, OtherLayoutPolicy, OtherAccessor>& other);
```

- *Constraints:*
 - `is_convertible_v<OtherLayoutPolicy::template mapping<OtherExtents>, mapping_type>` is true;

- `is_convertible_v<OtherAccessor, Accessor>` is true;
- `is_convertible_v<OtherAccessor::pointer, pointer>` is true;
- `is_convertible_v<OtherExtents, extents_type>` is true; and
- For all `r` in the range `[0, rank())`, if `other.static_extent(r) != dynamic_extent` && `static_extent(r) != dynamic_extent` is true, then `other.static_extent(r) == static_extent(r)` is true.
- *Requires:*
 - For all `r` in the range `[0, rank())`, if `other.static_extent(r) == dynamic_extent` || `static_extent(r) == dynamic_extent` is true, then `other.extent(r) == extent(r)` is true.
- *Effects:*
 - Initializes `ptr_` with `other.ptr_`,
 - Initializes `map_` with `other.map_`, and
 - Initializes `acc_` with `other.acc_`.
- *Ensures:*
 - `extents() == Extents(other.extents())` is true, and
 - `mapping() == mapping_type(other.mapping())` is true.
- *Throws:* Nothing.

```
template<class OtherElementType, class OtherExtents, class OtherLayoutPolicy, class OtherAccessor>
constexpr basic_mdspan& operator=(const basic_mdspan<OtherElementType, OtherExtents, OtherLayoutPolicy, OtherAccessor>& other);
```

- *Constraints:*
 - `is_assignable_v<mapping_type, OtherLayoutPolicy::template mapping<OtherExtents>>` is true;
 - `is_assignable_v<Accessor, OtherAccessor>` is true;
 - `is_assignable_v<pointer, OtherAccessor::pointer>` is true;
 - `OtherExtents::rank() == rank()` is true; and
 - For all `r` in the range `[0, rank())`, if `other.static_extent(r) != dynamic_extent` && `static_extent(r) != dynamic_extent` is true, then `other.static_extent(r) == static_extent(r)` is true.
- *Requires:*
 - For all `r` in the range `[0, rank())`, if `other.static_extent(r) == dynamic_extent` || `static_extent(r) == dynamic_extent` is true, then `other.extent(r) == extent(r)` is true.
- *Effects:*
 - Assigns `other.ptr_` to `ptr_`,
 - Assigns `other.map_` to `map_`, and
 - Assigns `other.acc_` to `acc_`.
- *Ensures:*
 - `extents() == Extents(other.extents())` is true, and
 - `mapping() == mapping_type(other.mapping())` is true.
- *Throws:* Nothing.

[mdspan.basic.mapping]

```
constexpr reference operator[](index_type i) const;
```

- *Constraints:* `rank() == 1` is true.
- *Requires:* `acc_.access(ptr_, map_(i))` shall be valid.
- *Effects:* Equivalent to: `return (*this)(i);`.

```
template<class... IndexType>  
constexpr reference operator()(IndexType... indices) const;
```

- *Constraints:*
 - `(is_convertible_v<IndexType, index_type> && ...)` is true, and
 - `sizeof...(IndexType) == rank()` is true.
- *Requires:* `acc_.access(ptr_, map_(indices...))` shall be valid.
- *Effects:* Equivalent to: `return acc_.access(ptr_, map_(indices...));`.
- *Throws:* Nothing.

```
template<class IndexType, size_t N>  
constexpr reference operator()(const array<IndexType, N>& indices) const;
```

- *Constraints:*
 - `is_convertible_v<IndexType, index_type>` is true, and
 - `rank() == N` is true.
- *Requires:* `acc_.access(ptr_, std::apply(map_, indices))` shall be valid.
- *Effects:* Equivalent to: `return std::apply(*this, indices);`.
- *Throws:* nothing.

```
accessor_type accessor() const;
```

- *Returns:* `acc_`.

26.7.3 basic_mdspan observers of the domain multidimensional index space [mdspan.basic.domobs]

```
static constexpr int rank() noexcept;
```

- *Returns:* `Extents::rank()`.

```
static constexpr int rank_dynamic() noexcept;
```

- *Returns:* `Extents::rank_dynamic()`.

```
static constexpr index_type static_extent(size_t r) noexcept;
```

- Returns: `Extents::static_extent(r)`.

```
constexpr Extents extents() const noexcept;
```

- Returns: `mapping().extents()`.

```
constexpr index_type extent(size_t r) const noexcept;
```

- Returns: `extents().extent(r)`.

```
constexpr index_type size() const noexcept;
```

- Returns: Product of `extent(r)` for all `r` in the range `[0, extents().rank())`.

```
constexpr index_type unique_size() const noexcept;
```

- Returns: The number of unique elements in the codomain. *[Note: If `mapping().is_unique()` is true, this is identical to `size()`. —end note]*

26.7.4.3 `basic_mdspan` observers of the codomain `[mdspan.basic.codomain]`

```
constexpr span<element_type> span() const noexcept;
```

- Effects: Equivalent to: `return span<element_type>(acc_.decay(ptr_), required_span_size());`.

```
constexpr pointer data() const noexcept;
```

- Returns: `ptr_`.

26.7.4.4 `basic_mdspan` observers of the mapping `[mdspan.basic.obs]`

```
static constexpr bool is_always_unique() noexcept;
```

- Returns: `mapping_type::is_always_unique()`.

```
static constexpr bool is_always_contiguous() noexcept;
```

- Returns: `mapping_type::is_always_contiguous()`.

```
static constexpr bool is_always_strided() noexcept;
```

- Returns: `mapping_type::is_always_strided()`.

```
constexpr mapping_type mapping() const noexcept;
```

- Returns: `map_`.

```
constexpr bool is_unique() const noexcept;
```

- Returns: `mapping().is_unique()`.

```
constexpr bool is_contiguous() const noexcept;
```

- Returns: `mapping().is_contiguous()`.

```
constexpr bool is_strided() const noexcept;
```

- Returns: `mapping().is_strided()`.

```
constexpr index_type stride(size_t r) const;
```

- Returns: `mapping().stride(r)`.

26.7. **subspan** [`mdspan.subspan`]

1. The `subspan` function creates a `basic_mdspan` with a domain that is a subset of the input `basic_mdspan`'s domain, and a codomain that is a subset of the input `basic_mdspan`'s codomain.
2. The `SliceSpecifier` template argument(s) and the corresponding value(s) of the arguments of `subspan` after `src` determine the subset of `src` that the return value views.


```

namespace std {
namespace experimental {
inline namespace fundamentals_v3 {

    // [mdspan.subspan], subspan creation
    template<class ElementType, class Extents, class LayoutPolicy,
            class AccessorPolicy, class... SliceSpecifiers>
    struct mdspan_subspan { // exposition only
        using extents_t = see below;
        using layout_t = see below;
        using type = basic_mdspan<ElementType, extents_t, layout_t,
                                typename AccessorPolicy::offset_policy>;
    };

    template<class ElementType, class Extents, class LayoutPolicy,
            class AccessorPolicy, class... SliceSpecifiers>
    using mdspan_subspan_t = // exposition only
        typename mdspan_subspan<ElementType, Extents, LayoutPolicy,
                                AccessorPolicy, SliceSpecifiers...>::type;

    template<class ElementType, class Extents, class LayoutPolicy,
            class AccessorPolicy, class... SliceSpecifiers>
    mdspan_subspan_t<ElementType, Extents, LayoutPolicy,
                    AccessorPolicy, SliceSpecifiers...>
    subspan(const basic_mdspan<ElementType, Extents, LayoutPolicy,
                            AccessorPolicy>& src,
            SliceSpecifiers ... slices) noexcept;

}}}

```

3. Let `sub` be the return value of `subspan(src, slices...)`. Denote the value of the r -th element of `slices...` by k_r , and denote the type of the r -th element of `slices...` by k_r .
4. Let k equal the number of k_r such that `is_convertible_v<  $k_r$ , pair<ptrdiff_t, ptrdiff_t>> || is_convertible_v<  $k_r$ , all_type>` is true.
5. Define `rank_map` as the length `integer_sequence` of `ptrdiff_t` consisting of the unique values of k_r , in increasing order, such that `is_convertible_v<  $k_r$ , pair<ptrdiff_t, ptrdiff_t>> || is_convertible_v<  $k_r$ , all_type>` is true.
6. Define the exposition-only `constexpr` function `get_seq<k>(seq)`, where k is an integer, as the function returning the k -th entry of the `std::integer_sequence` `seq`. We say that the integer r "is not in `rank_map`" when, for all k in the range `rank_map`, r does not equal `get_seq<k>(rank_map)`.
7. In the exposition-only `struct mdspan_subspan`,
 - The `extents_t` type alias is a specialization of `extents` [**mdspan.extents**], and is determined by the description of the semantics of `subspan` given below;
 - The `layout_t` type alias meets the requirements of a layout mapping policy [**mdspan.layout.reqs**] and is implementation defined; and
 - The `type` type alias specifies the type returned by `subspan`.

[Note: High-quality implementations will avoid `layout_stride` whenever possible, if the input layout is `layout_left` or `layout_right`. — end note]

8. Let `first` and `last` be exposition-only `array<ptrdiff_t, sizeof...(SliceSpecifier)>`, where `SliceSpecifier...` is the same parameter pack as `subspan`'s. For r in the range `[0, sizeof...(SliceSpecifier))`, define the values of `first[r]` and `last[r]` as follows, where `src` and `slices...` are the arguments of `subspan`:
 - If `is_convertible_v<  $k_r$ , ptrdiff_t>`, then `first[r]` equals k_r , and `last[r]` equals `first[r] + 1`;
 - Else, if `is_convertible_v<  $k_r$ , pair<ptrdiff_t, ptrdiff_t>>`, then `first[r]` equals `p.first`, and `last[r]` equals `p.second`, where `p` is the result of converting k_r to `pair<ptrdiff_t,`

`ptrdiff_t>;`

- Else, if `is_convertible_v< r, all_type>`, then `first[r] = 0`, and `last[r]` equals `src.extent(r)`.

- *Requires:*

- `sizeof(slices...)` equals `src.rank()`; and
- For all `r` in the range `[0, sizeof...(slices))`, `0 <= first[r] && first[r] < last[r] && last[r] <= src.extent(r)` is true.

- *Constraints:*

- `LayoutPolicy` is `layout_left`, `layout_right`, `layout_stride`, or any type in a possibly empty set of implementation-defined types, each of which meets the requirements of a layout mapping policy **[mdspan.layout.reqs]** *[Note: Valid and useful layout mapping policies exist, for which taking an arbitrary `subspan` does not make sense. — end note];* and
- For all `k` in the range `[0, sizeof...(slices))`, `is_convertible_v< k, ptrdiff_t> || is_convertible_v< k, pair<ptrdiff_t, ptrdiff_t>> || is_convertible_v< k, all_type>` is true.

- *Ensures:* All of the following:

- `sub.rank()` equals `.`
- `mdspan_subspan<ElementType, Extents, LayoutPolicy, AccessorPolicy, SliceSpecifiers...>::extents_t::rank()` equals `.`
- Let the pack `i...` denote a multidimensional index in the domain of `sub` such that `k < sub.extents(k)` for all `k` in the range `.`, where `k` denotes the `k`-th element of `i...`. Let the pack `j...` denote a multidimensional index in the domain of `src` where the `get_seq<k>(rank_map)`-th element of `j...` equals `first[get_seq<k>(rank_map)] + k` for all `k` in the range `.`, and the `r`-th element of `j...` equals `first[r]` for all `r` in the range `[0, src.rank())` such that `r` is not in `rank_map`. Then, `sub(i...)` and `src(j...)` refer to the same element.
- For all `k` in the range `.`, `sub.extent(k)` equals `last[get_seq<k>(rank_map)] - first[get_seq<k>(rank_map)]`.
- If `src.is_strided()`, then `sub.is_strided()` is true, and for all `k` in the range `.`, `sub.stride(k)` equals `src.stride(get_seq<k>(rank_map))`.
- For all `k` in the range `.`, let `rmk` equal `get_seq<k>(rank_map)`. If `src.static_extent(rmk)` does not equal `dynamic_extent` and `is_convertible_v< rmk, all_type>` is true, then `sub.static_extent(k)` equals `src.static_extent(rmk)`.

[Note: Example of `subspan` use:

```

// Create a mapping
typedef extents<3,dynamic_extent,7> Extents3D;
layout_right::template mapping<Extents3D> map_right(10);

// Allocate a basic_mdspan
int* ptr = new int[3*8*10];
basic_mdspan<int,Extents3D,layout_right> a(ptr,map_right);

// Initialize the span
for(int i0=0; i0<a.extent(0); i0++)
    for(int i1=0; i1<a.extent(1); i1++)
        for(int i2=0; i2<a.extent(2); i2++)
            a(i0,i1,i2) = 10000*i0+100*i1+i2;

// Create Subspan
auto a_sub = subspan(a,1,std::pair<int,int>(4,6),std::pair<int,int>(1,6));

// Print values of subspan
for(int i0=0; i0<a_sub.extent(0); i0++) {
    for(int i1=0; i1<a_sub.extent(1); i1++) {
        std::cout << a_sub(i0,i1) << " ";
    }
    std::cout << std::endl;
}

/* Output
10401 10402 10403 10404 10405
10501 10502 10503 10504 10505
*/

```

- end note]

§ 5. Next Steps

- Wording editing as per guidance from LWG.

§ 6. Related Work

[LEWG issue](#)

Previous paper:

- [\[N4355\]](#)

P0860 : Access Policy Generating Proxy Reference

The `reference` type may be a proxy for accessing an `element_type` object. For example, the `atomic AccessorPolicy` in **P0860** defines `AccessorPolicy::template accessor_type<T>::reference` to be `atomic_ref<T>` from **P0019**.

Related papers:

- **P0122** : span: bounds-safe views for sequences of objects The `mdspan` codomain concept of `span` is well-aligned with this paper.
- **P0367** : Accessors: The P0367 Accessors proposal includes polymorphic mechanisms for accessing the memory an object or span of objects. The `AccessorPolicy` extension point in this proposal is intended to include such memory access properties.
- **P0331** : Motivation and Examples for Multidimensional Array
- **P0332** : Relaxed Incomplete Multidimensional Array Type Declaration
- **P0454** : Wording for a Minimal `mdspan` Included proposed modification of `span` to better align `span` with `mdspan`.

- **P0546** : Preparing `span` for the future Proposed modification of `span`
- **P0856** : Restrict access property for `mdspan` and `span`
- **P0860** : atomic access policy for `mdspan`
- **P0900** : An Ontology of Properties for `mdspan`

§ References

§ Informative References

[N4355]

Carter Edwards. [Shared Multidimensional Arrays with Polymorphic Layout](https://wg21.link/n4355). 4 February 2015. URL: <https://wg21.link/n4355>

[P0900r0]

David S. Hollman. [An Ontology for Properties of mdspan](https://wg21.link/p0900r0). 12 February 2018. URL: <https://wg21.link/p0900r0>